PERSONAL FINANCE MANAGER

Software Design Document (SDD)

1. Introduction

The Personal Finance Manager is a console-based application designed to help users manage their financial activities efficiently. It allows users to track expenses, manage income, set budgets, and view summaries of their financial data.

The system is built using **TypeScript** and follows clean architecture principles to ensure maintainability, scalability, and readability.

2. Objective

The main objective of this application is to:

- Track income and expenses
- Categorize financial data
- Manage budgets effectively
- Provide financial insights through summaries

3. Scope of the System

The system includes the following functionalities:

- Expense Management
- Income Management
- Budget Management
- Summary Reporting
- Data Persistence using a relational database

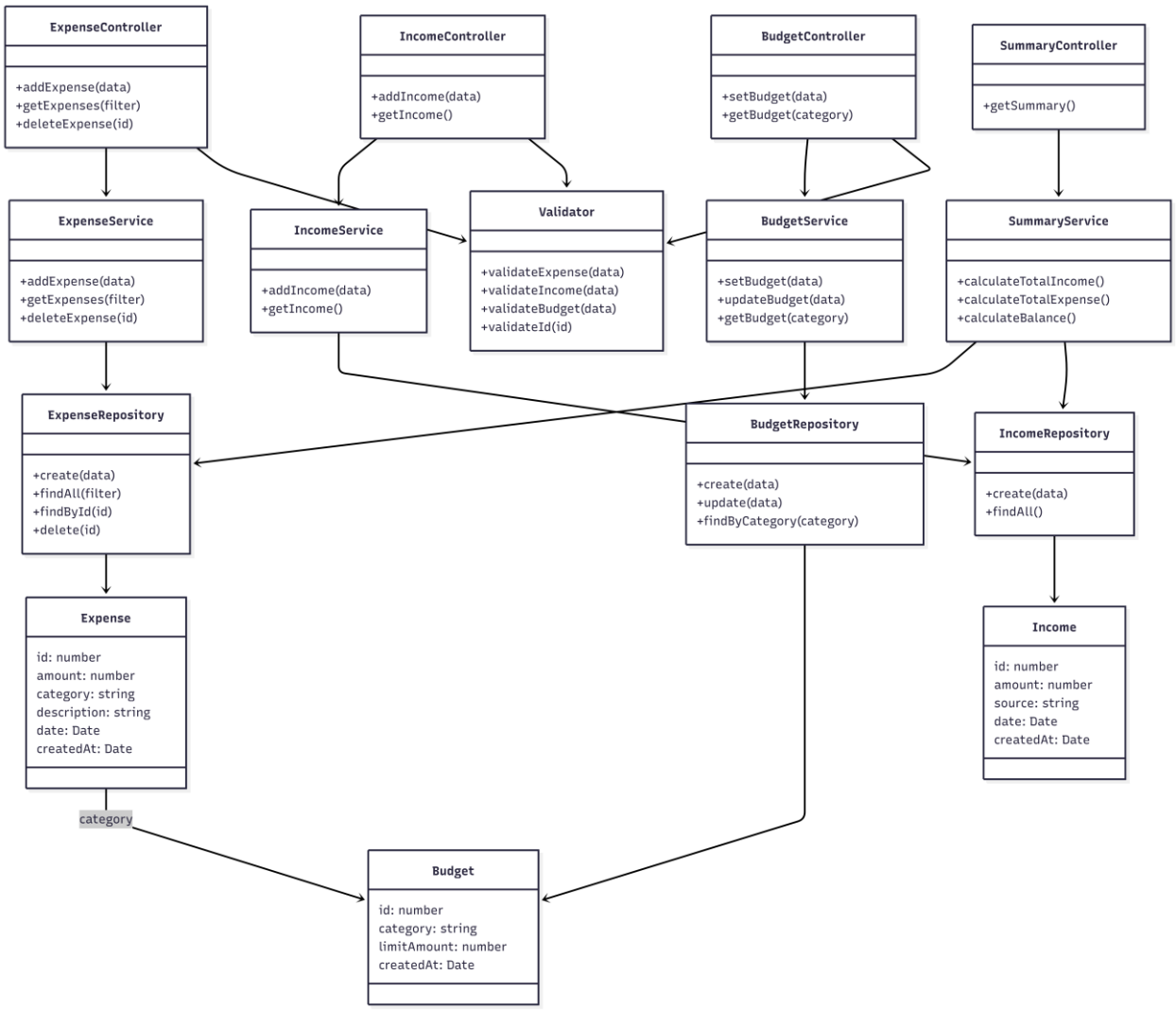
- Input validation and error handling

4. System Architecture

The application follows a **layered architecture**:

- CLI Layer (Console Interaction)
- Controller Layer
- Service Layer
- Repository Layer
- Database Layer

Class Diagram



5. Functional Requirements

5.1 Expense Management

- Add expense
- View all expenses
- Filter expenses by category/date
- Delete expense

5.2 Income Management

- Add income
- View income

5.3 Budget Management

- Set budget per category
- Update budget
- Track expenses against budget

5.4 Summary

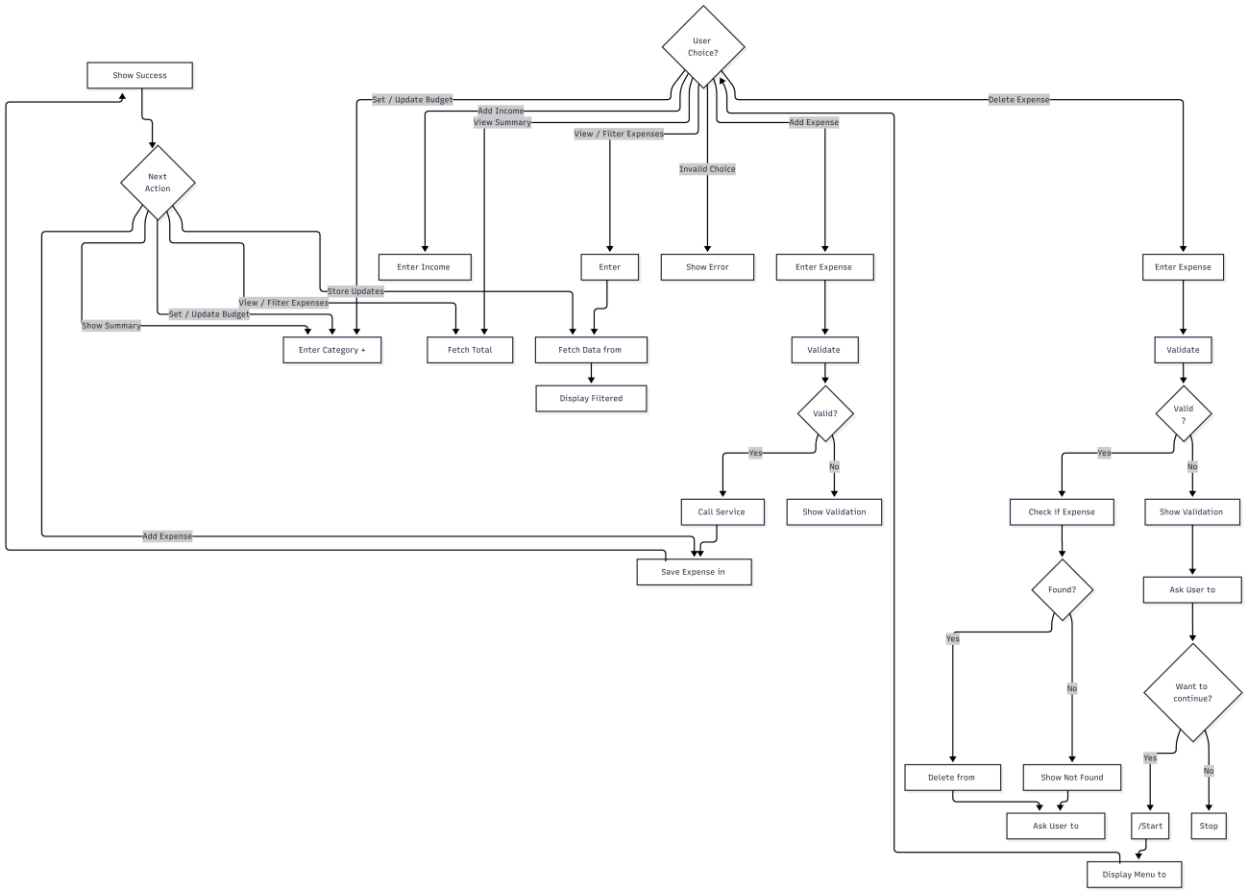
- Total income
- Total expenses
- Balance calculation
- Category-wise spending

6. System Workflow

The application works as follows:

1. User selects an operation from the console menu
2. Input is provided by the user
3. Validation is performed
4. Controller processes the request
5. Service layer executes business logic
6. Repository interacts with database
7. Result is displayed to the user

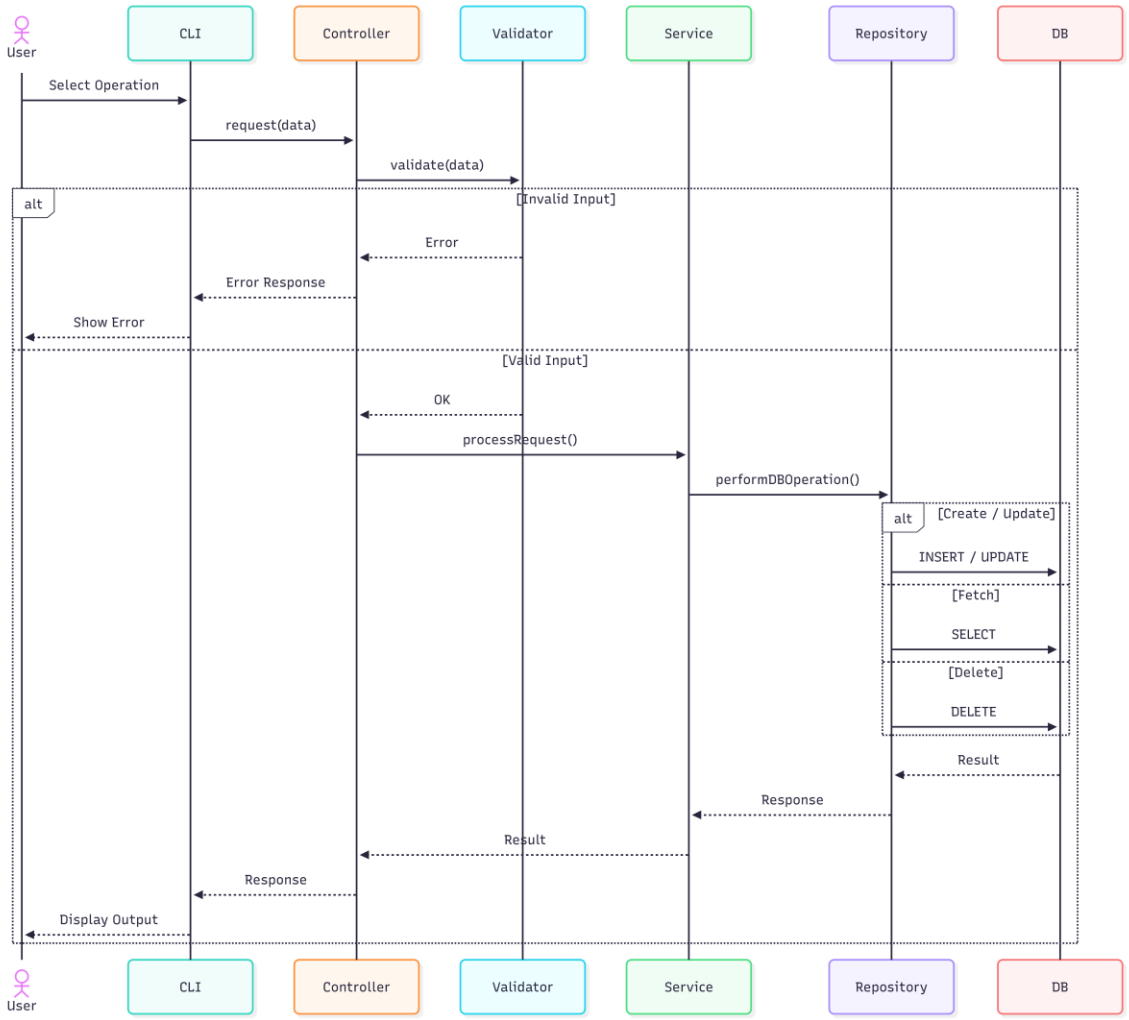
Flow Diagram (Activity Diagram)



7. Sequence of Operations

This diagram illustrates how different components interact during operations such as adding expense, fetching data, and generating summary.

Sequence Diagram



8. Database Design

The system uses a relational database to store all financial data.

Tables:

Expense

- id (Primary Key)
- amount
- description
- date
- categoryId (Foreign Key)

Income

- id (Primary Key)
- amount
- source
- date

Budget

- id (Primary Key)
- categoryId (Foreign Key)
- limitAmount

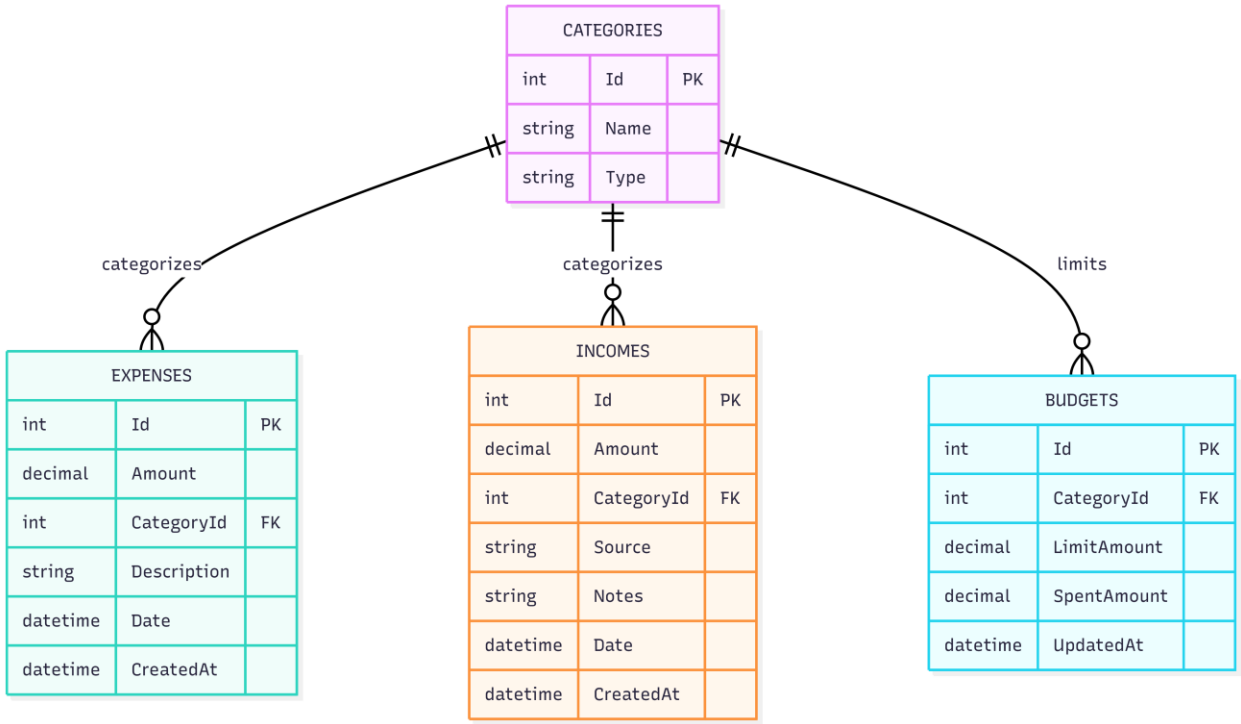
Category

- id (Primary Key)
- name (Unique)

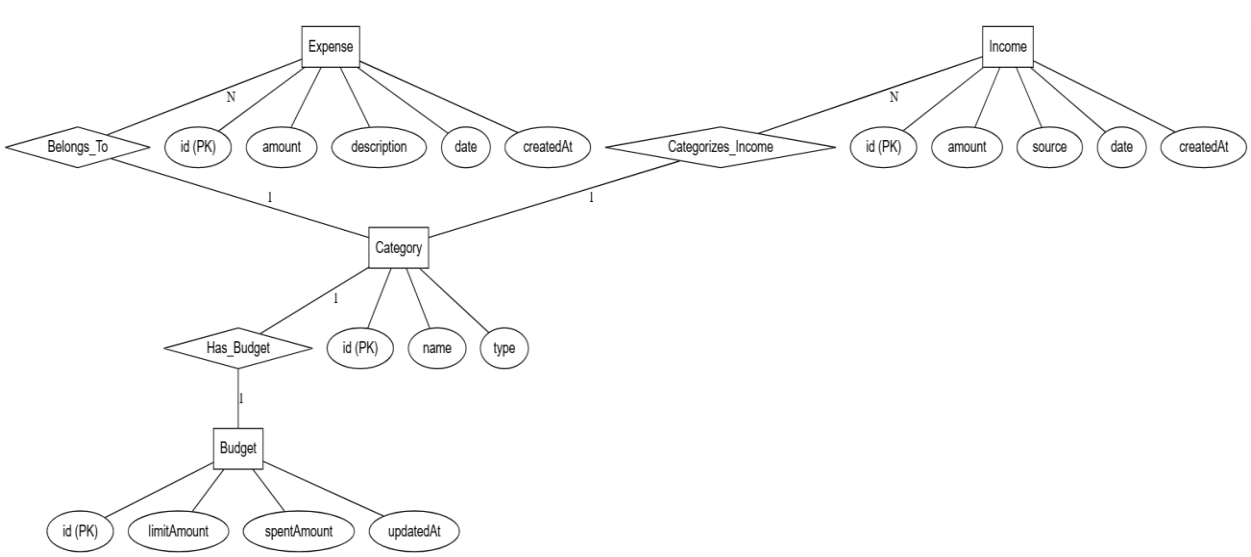
Relationships:

- Category → Expense (1:N)
- Category → Budget (1:1)

Database Diagram



ER Diagram



9. Class Design Overview

The system is structured into multiple classes:

Entities:

- Expense
- Income
- Budget
- Category

Controllers:

- ExpenseController
- IncomeController
- BudgetController
- SummaryController

Services:

- ExpenseService
- IncomeService
- BudgetService
- SummaryService

Repositories:

- ExpenseRepository
- IncomeRepository
- BudgetRepository
- CategoryRepository

Validator:

- Handles input validation

10. Validation & Error Handling

Validation Rules:

- Amount must be positive
- Date must be valid
- Category must exist
- ID must be valid

Error Handling:

- Invalid input messages
- Database failure handling
- Record not found handling

11. Data Persistence

- Uses a relational database
- Ensures data is stored across application restarts
- Supports CRUD operations

12. Summary Calculation Logic

- Total Income = SUM(Income.amount)
- Total Expense = SUM(Expense.amount)
- Balance = Income - Expense

13. Non-Functional Requirements

- Maintainability

- Scalability
- Performance efficiency
- Readability
- Modularity

14. Design Principles Used

- Separation of Concerns
- Single Responsibility Principle
- Modular Design
- Clean Code Practices

15. Test Cases

Expense Management

Test Case	Input	Expected Output
Valid Expense	500, Food	Added Successfully
Invalid Amount	-100	Error
Delete Expense	Valid ID	Deleted
Delete Invalid ID	999	Error

Income Management

Test Case	Input	Expected
Add Income	10000	Success
Invalid Income	0	Error

Budget Management

Test Case	Input	Expected
Set Budget	Food: 5000	Success
Exceed Budget	Expense > limit	Warning

Filtering

Scenario	Expected
Filter by category	Only matching records
Filter by date	Correct subset

16. Constraints

- Console-based interface
- No hardcoding
- Must handle edge cases
- Proper validation required

17. Assumptions

- Single-user system
- Categories are manageable
- Database is locally available

18. Conclusion

The Personal Finance Manager is designed using clean architecture and modular principles. It provides a scalable and maintainable solution for managing financial data efficiently through a console interface.